Assignment 2: Pattern Mining

Deadline: 20 April 2026

Name: YOUR_NAME

Matriculation Number: YOUR_MATRICULATION_NUMBER

Packages: Pandas, Mlxtend

Submission Instructions: Upload your Jupyter notebook and a PDF export (with results) on Moodle. Go then to the corresponding checkmark list to indicate which tasks you have completed and feel confident to explain in class. The checkmark list will be the basis for grading. If you indicate on Moodle to be able to explain a solution to a task but then fail to appropriately explain the submitted solution, including the motivation for individual steps, during the assignment review, you will fail the course. If you indicate that you solved a specific task but the task was clearly not solved, you will fail the course.

Write your solutions in the code cells for the different tasks. You may also add additional code cells as well as markdown cells if you want to write down additional explanations, observations, or assumptions.

There are also questions that require you to write textual answers into markdown cells.

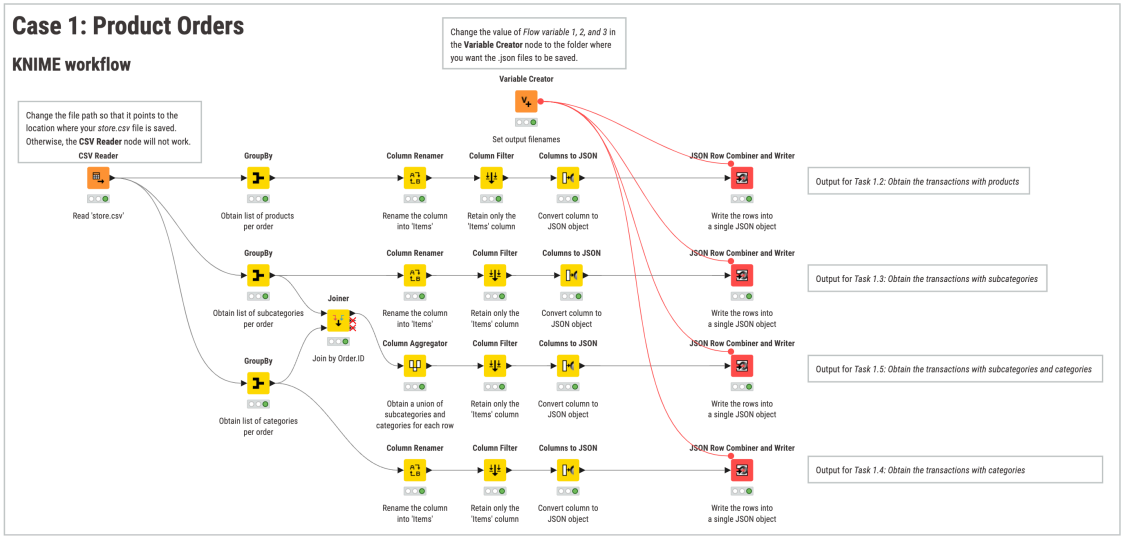
Note: The tables shown in the PDF export of this notebook on Moodle are only illustrative. You may obtain different numbers.

Case 1: Product Orders

Files: store.csv

You have a dataset of orders along with product and customer information. Each order comprises multiple items. Each row in the dataset represents an order item. You should conduct a market basket analysis by mining for frequent patterns and association rules.

To be able to mine for frequent patterns and association rules, you have to first preprocess the data using pandas to obtain transactions. The following KNIME workflow illustrates the preprocessing steps. If you cannot preprocess the data using pandas, you may also use the output of the KNIME workflow as the starting point for association rule mining.



[1]:
```python
# Imports
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
```

Task 1.1: Read the file

Use pandas to read the CSV file. Choose the right encoding and make *Category* a string column. Show the first couple of rows of the dataframe.

[2]:
```python

```

[2]:
```
   Row.ID         Order.ID Order.Date Ship.Date  Ship.Mode Customer.ID  \
0   42433      AG-2011-2040   1/1/2011  6/1/2011          4    TB-11280
1   22253     IN-2011-47883   1/1/2011  8/1/2011          4    JH-15985
2   48883      HU-2011-1220   1/1/2011  5/1/2011          3      AT-735
3   11731   IT-2011-3647632   1/1/2011  5/1/2011          3    EM-14140
4   22255     IN-2011-47883   1/1/2011  8/1/2011          4    JH-15985

     Customer.Name  Segment         City            State  … Category  \
0  Toby Braunhardt        1  Constantine      Constantine  …        2
1      Joseph Holt        1  Wagga Wagga  New South Wales  …        2
2    Annie Thurman        1     Budapest         Budapest  …        2
3     Eugene Moren        3    Stockholm        Stockholm  …        2
4      Joseph Holt        1  Wagga Wagga  New South Wales  …        1

   Sub.Category                 Product.Name    Sales Quantity Discount  \
0       Storage          Tenex Lockers, Blue  408.300        2      0.0
1      Supplies     Acme Trimmer, High Speed  120.366        3      0.1
2       Storage      Tenex Box, Single Width   66.120        4      0.0
```

```
3         Paper  Enermax Note Cards, Premium   44.865        3      0.5
4   Furnishings   Eldon Light Bulb, Duo Pack  113.670        5      0.1

    Profit Shipping.Cost  Status  Order.Priority
0  106.140         35.46       1               4
1   36.036          9.72       1               4
2   29.640          8.17       1               2
3  -26.055          4.82       1               2
4   37.770          4.70       1               4

[5 rows x 25 columns]
```

Task 1.2: Obtain the transactions with products

Use pandas to group the rows by *Order.ID* and aggregate the *Product.ID* values into a list for each order. For convenience, rename the *Product.ID* column to *Items* and drop the *Order.ID* column. Show the first couple of rows of the resulting dataframe.

[3]:

```
```

[3]:

```
                                            Items
0               [OFF-FEL-10001405, TEC-EPS-10004171]
1               [FUR-BUS-10003055, OFF-ACC-10004278]
2               [OFF-TEN-10002817, OFF-STI-10000114]
3                                  [OFF-ROG-10001340]
4  [TEC-MOT-10001535, OFF-ELD-10002297, OFF-BIC-1…
```

Task 1.3: Obtain the transactions with subcategories

Use pandas to group the rows by *Order.ID* and aggregate the *Sub.Category* values into a list for each order. For convenience, rename the *Sub.Category* column to *Items* and drop the *Order.ID* column. Show the first couple of rows of the resulting dataframe.

[4]:

```
```

[4]:

```
                                            Items
0                                [Storage, Machines]
1                              [Bookcases, Fasteners]
2                                [Storage, Supplies]
3                                          [Storage]
4  [Phones, Storage, Art, Storage, Binders, Binders]
```

Task 1.4: Obtain the transactions with categories

Use pandas to group the rows by *Order.ID* and aggregate the *Category* values into a list for each order. For convenience, rename the *Category* column to *Items* and drop the *Order.ID* column. Show the first couple of rows of the resulting dataframe.

[5]:

```
```

[5]:

```
                Items
0              [2, 3]
1              [1, 2]
2              [2, 2]
3                 [2]
4  [3, 2, 2, 2, 2, 2]
```

Task 1.5: Obtain the transactions with subcategories and categories

Use pandas to obtain transactions with a *set* of categories and subcategories instead of the individual categories/subcategories. For convenience, rename your column containing the *set* of categories and subcategories to *Items* and drop the other columns. Show the first couple of rows of the resulting dataframe.

[6]:

```
```

[6]:

```
                                   Items
0              {2, 3, Machines, Storage}
1           {2, 1, Bookcases, Fasteners}
2                 {2, Supplies, Storage}
3                           {2, Storage}
4  {Storage, 2, Art, Binders, 3, Phones}
```

Task 1.6: Transform the transactions with products into the required format

Obtain a one-hot encoded representation of the transactions with products in the format required by the Mlxtend framework. Use the one-hot encoded representation to create a dataframe with the respective products as columns and show the first couple of rows.

[7]:

```
```

[7]:

```
   FUR-ADV-10000002  FUR-ADV-10000108  FUR-ADV-10000183  FUR-ADV-10000188  \
0             False             False             False             False
1             False             False             False             False
2             False             False             False             False
3             False             False             False             False
4             False             False             False             False

   FUR-ADV-10000190  FUR-ADV-10000571  FUR-ADV-10000600  FUR-ADV-10000847  \
0             False             False             False             False
1             False             False             False             False
2             False             False             False             False
3             False             False             False             False
4             False             False             False             False

   FUR-ADV-10001283  FUR-ADV-10001440  …  TEC-STA-10003330  \
```

```
0             False             False  …             False
1             False             False  …             False
2             False             False  …             False
3             False             False  …             False
4             False             False  …             False

   TEC-STA-10003386  TEC-STA-10003447  TEC-STA-10003550  TEC-STA-10003925  \
0             False             False             False             False
1             False             False             False             False
2             False             False             False             False
3             False             False             False             False
4             False             False             False             False

   TEC-STA-10004181  TEC-STA-10004536  TEC-STA-10004542  TEC-STA-10004834  \
0             False             False             False             False
1             False             False             False             False
2             False             False             False             False
3             False             False             False             False
4             False             False             False             False

   TEC-STA-10004927
0             False
1             False
2             False
3             False
4             False

[5 rows x 10292 columns]
```

Task 1.7: Transform the transactions with subcategories into the required format

Obtain a one-hot encoded representation of the transactions with subcategories in the format required by the Mlxtend framework. Use the one-hot encoded representation to create a dataframe with the respective subcategories as columns and show the first couple of rows.

[8]:

```
```

[8]:

```
   Accessories  Appliances    Art  Binders  Bookcases  Chairs  Copiers  \
0        False       False  False    False      False   False    False
1        False       False  False    False       True   False    False
2        False       False  False    False      False   False    False
3        False       False  False    False      False   False    False
4        False       False   True     True      False   False    False

   Envelopes  Fasteners  Furnishings  Labels  Machines  Paper  Phones  \
0      False      False        False   False      True  False   False
1      False       True        False   False     False  False   False
2      False      False        False   False     False  False   False
```

```
3      False      False        False   False     False  False   False
4      False      False        False   False     False  False    True

   Storage  Supplies  Tables
0     True     False   False
1    False     False   False
2     True      True   False
3     True     False   False
4     True     False   False
```

Task 1.8: Transform the transactions with categories into the required format

Obtain a one-hot encoded representation of the transactions with categories in the format required by the Mlxtend framework. Use the one-hot encoded representation to create a dataframe with the respective categories as columns and show the first couple of rows.

[9]:

```
```

[9]:

```
       1     2      3
0  False  True   True
1   True  True  False
2  False  True  False
3  False  True  False
4  False  True   True
```

Task 1.9: Transform the transactions with subcategories and categories into the required format

Obtain a one-hot encoded representation of the transactions with subcategories and categories in the format required by the Mlxtend framework. Use the one-hot encoded representation to create a dataframe with the respective categories and subcategories as columns and show the first couple of rows.

[10]:

```
```

[10]:

```
       1     2      3  Accessories  Appliances    Art  Binders  Bookcases  \
0  False  True   True        False       False  False    False      False
1   True  True  False        False       False  False    False       True
2  False  True  False        False       False  False    False      False
3  False  True  False        False       False  False    False      False
4  False  True   True        False       False   True     True      False

   Chairs  Copiers  Envelopes  Fasteners  Furnishings  Labels  Machines  \
0   False    False      False      False        False   False      True
1   False    False      False       True        False   False     False
2   False    False      False      False        False   False     False
3   False    False      False      False        False   False     False
4   False    False      False      False        False   False     False
```

```
   Paper  Phones  Storage  Supplies  Tables
0  False   False     True     False   False
1  False   False    False     False   False
2  False   False     True      True   False
3  False   False     True     False   False
4  False    True     True     False   False
```

Task 1.10: Mine for frequent itemsets with products

Use the Mlxtend framework to mine for frequent itemsets in the transactions with products having a minimum support of 0.1 %. Add a column *length* indicating the length of the itemset and a column *support_count* indicating the support count of the itemset. Sort the dataframe by the support count in descending order and show all rows.

[11]:

```
[11]:     support                      itemsets  length  support_count
0  0.001398  frozenset({OFF-AR-10003651})       1             35
1  0.001238  frozenset({OFF-AR-10003829})       1             31
2  0.001198  frozenset({OFF-BI-10003708})       1             30
3  0.001158  frozenset({OFF-BI-10002799})       1             29
4  0.001118  frozenset({FUR-CH-10003354})       1             28
5  0.001078  frozenset({OFF-BI-10002570})       1             27
```

Task 1.11: Mine for frequent itemsets and association rules with subcategories

Use the Mlxtend framework to mine for frequent itemsets in the transactions with subcategories having a minimum support of 0.5 %. Add a column *length* indicating the length of the itemset and a column *support_count* indicating the support count of the itemset. Sort the dataframe by the support count in descending order and show the first couple of rows.

Try to find association rules with a minimum lift of 1 and sort the rules by Kulczynski measure in descending order. Show the first couple of rows of the dataframe, limited to the columns *antecedents*, *consequents*, *support*, *confidence*, *lift*, and *kulczynski*.

Finally, interpret the interestingness of the found rules. Write your textual answer in the markdown cell below.

[12]:

```
[12]:     support              itemsets  length  support_count
0  0.215378  frozenset({Binders})       1           5392
1  0.181106  frozenset({Storage})       1           4534
2  0.174396      frozenset({Art})       1           4366
3  0.129179    frozenset({Paper})       1           3234
4  0.127302   frozenset({Chairs})       1           3187
```

[13]:

[13]:

```
                    antecedents                    consequents   support  \
0              frozenset({Art})           frozenset({Storage})  0.033273
1          frozenset({Storage})               frozenset({Art})  0.033273
2          frozenset({Binders})   frozenset({Phones, Storage})  0.006870
3  frozenset({Phones, Storage})           frozenset({Binders})  0.006870
4          frozenset({Binders})  frozenset({Furnishings, Art})  0.005992

   confidence      lift  kulczynski
0    0.190792  1.053483    0.187258
1    0.183723  1.053483    0.187258
2    0.031899  1.489915    0.176397
3    0.320896  1.489915    0.176397
4    0.027819  1.491324    0.174509
```

How do you interpret the interestingness of the found rules?

-> PUT YOUR ANSWER HERE <-

Task 1.12: Mine for frequent itemsets and association rules with categories

Use the Mlxtend framework to mine for frequent itemsets in the transactions with categories having a minimum support of 1.0 %. Add a column *length* indicating the length of the itemset and a column *support_count* indicating the support count of the itemset. Sort the dataframe by the support count in descending order and show the first couple of rows.

Try to find association rules with a minimum lift of 1 and sort the rules by Kulczynski measure in descending order. Show the first couple of rows of the dataframe, limited to the columns *antecedents*, *consequents*, *support*, *confidence*, *lift*, and *kulczynski*.

Finally, interpret the interestingness of the found rules. Write your textual answer in the markdown cell below.

[14]:

```
```

[14]:

```
    support           itemsets  length  support_count
0  0.759057     frozenset({2})       1          19003
1  0.333693     frozenset({3})       1           8354
2  0.327342     frozenset({1})       1           8195
3  0.201718  frozenset({2, 3})       2           5050
4  0.192051  frozenset({2, 1})       2           4808
```

[15]:

```
```

[15]:

```
         antecedents        consequents   support  confidence      lift  \
0  frozenset({2, 1})     frozenset({3})  0.069063    0.359609  1.077665
1     frozenset({3})  frozenset({2, 1})  0.069063    0.206967  1.077665
2  frozenset({2, 3})     frozenset({1})  0.069063    0.342376  1.045929
3     frozenset({1})  frozenset({2, 3})  0.069063    0.210982  1.045929

   kulczynski
```

```
0    0.283288
1    0.283288
2    0.276679
3    0.276679
```

How do you interpret the interestingness of the found rules?

-> PUT YOUR ANSWER HERE <-

Task 1.13: Mine for frequent itemsets and association rules with subcategories and categories

Use the Mlxtend framework to mine for frequent itemsets in the transactions with subcategories and categories having a minimum support of 0.5 %. Add a column *length* indicating the length of the itemset and a column *support_count* indicating the support count of the itemset. Sort the dataframe by the support count in descending order and show the first and lasat couple of rows.

Try to find association rules with a minimum lift of 1 and sort the rules by Kulczynski measure in descending order. Show the first couple of rows of the dataframe, limited to the columns *antecedents*, *consequents*, *support*, *confidence*, *lift*, and *kulczynski*.

[16]:

```
    support                 itemsets  length  support_count
0  0.759057           frozenset({2})       1          19003
1  0.333693           frozenset({3})       1           8354
2  0.327342           frozenset({1})       1           8195
3  0.215378     frozenset({Binders})       1           5392
4  0.215378  frozenset({2, Binders})       2           5392

      support                                     itemsets  length  \
818  0.005033  frozenset({Furnishings, 2, 1, 3, Chairs})       5
819  0.005033        frozenset({2, 1, Chairs, Machines})       4
820  0.005033     frozenset({2, Furnishings, Chairs, 3})       4
821  0.005033     frozenset({2, Machines, 1, 3, Chairs})       5
822  0.005033   frozenset({2, Labels, Binders, Storage})       4

     support_count
818            126
819            126
820            126
821            126
822            126
```

[17]:

[17]:

```
            antecedents               consequents   support  confidence  \
0       frozenset({1})       frozenset({Chairs})  0.127302    0.388896
1  frozenset({Chairs})            frozenset({1})  0.127302    1.000000
2  frozenset({Phones})            frozenset({3})  0.125145    1.000000
```

```
3       frozenset({3})       frozenset({Phones})  0.125145    0.375030
4       frozenset({1})  frozenset({Furnishings})  0.118434    0.361806

       lift  kulczynski
0  3.054912    0.694448
1  3.054912    0.694448
2  2.996768    0.687515
3  2.996768    0.687515
4  3.054912    0.680903
```

Task 1.14: Get mappings from subcategories to categories

Create a map (dictionary) of subcatgories to categories from the base dataframe where subcategories are keys and categories are values.

[18]:

```
{'Accessories': '3', 'Appliances': '2', 'Art': '2', 'Binders': '2', 'Bookcases':
'1', 'Chairs': '1', 'Copiers': '3', 'Envelopes': '2', 'Fasteners': '2',
'Furnishings': '1', 'Labels': '2', 'Machines': '3', 'Paper': '2', 'Phones': '3',
'Storage': '2', 'Supplies': '2', 'Tables': '1'}
```

Task 1.15: Remove trivial rules

Use the mappings from subcategories to categories, i.e., the dictionary from Task 1.14, to filter out trivial rules where the category of a subcategory in the antecedents is in the consequents, and vice versa. For example, a rule *Phones, Chairs => 3* is a trivial rule because *Phones* belongs to category *3*.

Sort the rules by Kulczynski measure in descending order and show the first couple of rows of the dataframe, limited to the columns *antecedents*, *consequents*, *support*, *confidence*, *lift*, and *kulczynski*.

Finally, interpret the interestingness of the found rules. Write your textual answer in the markdown cell below.

[19]:

[19]:

```
                              antecedents  \
0  frozenset({Furnishings, 1, 3, Chairs})
1     frozenset({Furnishings, Chairs, 3})
2                          frozenset({2})
3                          frozenset({2})
4                          frozenset({2})

                              consequents   support  confidence      lift  \
0                          frozenset({2})  0.005033    0.818182  1.077892
1                          frozenset({2})  0.005033    0.818182  1.077892
2  frozenset({Furnishings, 1, 3, Chairs})  0.005033    0.006631  1.077892
3     frozenset({Furnishings, Chairs, 3})  0.005033    0.006631  1.077892
```

```
4     frozenset({Bookcases, Accessories})  0.007629    0.010051  1.022878

   kulczynski
0    0.412406
1    0.412406
2    0.412406
3    0.412406
4    0.393237
```

How do you interpret the interestingness of the found rules?

-> PUT YOUR ANSWER HERE <-

Case 2: Health Records

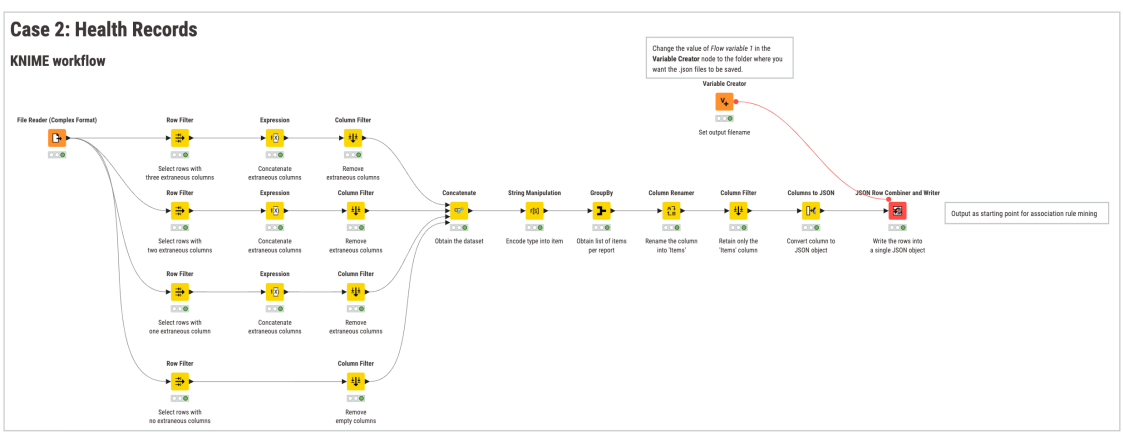
Files: health.csv

You have a dataset of health reports. Each report consists of multiple entries, which can be

- the diagnosis of a condition (type = 'C'),
- the observation of a symptom (type = 'S'),
- a prescribed treatment (type = 'T'),
- a note by the acting medical doctor (type = 'N'),
- or a weather observation (type = 'W').

You should mine for frequent patterns and association rules between conditions, symptoms, treatments, etc.

To be able to mine for frequent patterns and association rules, you have to first preprocess the data using pandas to obtain transactions. The following KNIME workflow illustrates the preprocessing steps. If you cannot preprocess the data using pandas, you may also use the output of the KNIME workflow as the starting point for association rule mining.



[20]:
```python
# Imports
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
```

```
from mlxtend.frequent_patterns import association_rules
```

Task 2.1: Write a function to handle bad lines

Write a function **handle_bad_line** that takes a list of string values as input parameter and returns a (repaired) list of string values or *None* as output.

In case of a line with more than seven columns, this function should be called to concatenate all string values after the seventh entry into a single string, using commas to separate the original values. The function should then return a list of string values with exactly seven entries, or *None* if that is not possible.

[1]:
```
# Define a custom function to handle bad lines
def handle_bad_line(bad_line: list) -> list | None:
    raise NotImplementedError('TOOD: Implement this function')
```

[22]:
```
# Exemplary bad line
original_list = ['this', 'list', 'has', 'too', 'many', 'entries', '!', '!', '!']

print('Example of how the function should processes a bad line:')
print(f'\t- Original list (bad line) with length {len(original_list)}:
 ↪\n\t{original_list}')

repaired_list = handle_bad_line(original_list)
print(f'\t- Repaired list (processed bad line by function) with length␣
 ↪{len(repaired_list)}:\n\t{repaired_list}')
```

```
Example of how the function should processes a bad line:
        - Original list (bad line) with length 9:
        ['this', 'list', 'has', 'too', 'many', 'entries', '!', '!', '!']
        - Repaired list (processed bad line by function) with length 7:
        ['this', 'list', 'has', 'too', 'many', 'entries', '!,!,!']
```

Task 2.2: Read the file

Read the CSV file into a pandas dataframe. The file contains *bad lines*, i.e., lines with more columns than expected. Use the *on_bad_lines* parameter of the **read_csv** function to handle those bad lines. If you have implemented the function **handle_bad_line** (Task 2.1), use it to repair the bad lines. Otherwise, you can simply skip the bad lines by setting the *on_bad_lines* parameter to 'skip'. Show the first couple of rows of the dataframe.

[23]:
```

```

[23]:
```
   Unnamed: 0                                user        date  reportid type  \
0      277224  QEVuQwEA///txlSWtDIyQTLnQ241zw==  2016-10-04     16906    C
1      277225  QEVuQwEA///txlSWtDIyQTLnQ241zw==  2016-10-04     16906    C
2      277226  QEVuQwEA///txlSWtDIyQTLnQ241zw==  2016-10-04     16906    S
3      277227  QEVuQwEA///txlSWtDIyQTLnQ241zw==  2016-10-04     16906    S
4      277228  QEVuQwEA///txlSWtDIyQTLnQ241zw==  2016-10-04     16906    T
```

```
                name       value
0          stiffness           4
1    herniated disk            2
2            fatigue           2
3    blood in urine            0
4  ferrous fumarate  1.0 tablet
```

Task 2.3: Encode type into name

Extend the dataframe with an *Items* column where the *type* is concatenated with the *name* to encode the type (symptom, condition, treatment, node, or weather observation) into the item name. The result of the concatenation should be in the format "type/name", e.g., "C/headache". Ensure that the *Items* column does not contain any NaN values. Drop all rows where the *Items* column contains NaN values.

Then group the dataframe by *reportid* and aggregate the items into a list. Show the first couple of rows of the resulting dataframe.

[24]:

```
```

[24]:

```
                                                Items
0                              [C/ulcerative colitis]
1                              [C/ulcerative colitis]
2  [C/pain in left upper arm felt like i was gett…
3                                  [C/crohns disease]
4  [C/gastroparesis, S/nausea, T/zofran, N/tired,…
```

Task 2.4: Transform the transactions into the required format

Obtain a one-hot encoded representation of the transactions in the format required by the Mlxtend framework. Use the one-hot encoded representation to create a dataframe with the respective column names and show the first couple of rows.

[25]:

```
```

[25]:

```
   C/abdominal aortic aneurysm  C/abdominal distention  C/abdominal migraine  \
0                        False                   False                 False
1                        False                   False                 False
2                        False                   False                 False
3                        False                   False                 False
4                        False                   False                 False

   C/abdominal migraines  C/abdominal pain  C/abdominal pains  \
0                  False             False              False
1                  False             False              False
2                  False             False              False
3                  False             False              False
4                  False             False              False
```

```
   C/absence seizure  C/absent seizures  C/ac ligament injury  \
0              False              False                 False
1              False              False                 False
2              False              False                 False
3              False              False                 False
4              False              False                 False

   C/acanthamoeba keratitis  …  T/zophran odt  T/zopiclone  T/zyrtec  \
0                     False  …          False        False     False
1                     False  …          False        False     False
2                     False  …          False        False     False
3                     False  …          False        False     False
4                     False  …          False        False     False

   T/zzzquil  W/humidity  W/icon  W/precip intensity  W/pressure  \
0      False       False   False               False       False
1      False       False   False               False       False
2      False        True    True                True        True
3      False       False   False               False       False
4      False       False   False               False       False

   W/temperature max  W/temperature min
0              False              False
1              False              False
2               True               True
3              False              False
4              False              False

[5 rows x 22615 columns]
```

Task 2.5: Obtain frequent itemsets

Use the Mlxtend framework to obtain frequent itemsets with a minimum support of 5 %. Add a column *support_count* indicating the support count of the itemset. Sort the dataframe by the support count in descending order and show the first and last couple of rows.

[26]:

```
    support                                          itemsets  support_count
0  0.372255                            frozenset({S/fatigue})          18663
1  0.328034                           frozenset({S/headache})          16446
2  0.294186                frozenset({W/pressure, W/humidity})          14749
3  0.294186  frozenset({W/precip intensity, W/temperature m…          14749
4  0.294186  frozenset({W/temperature min, W/temperature ma…          14749

      support                                          itemsets  \
693  0.050723                   frozenset({N/stressed, N/tired})
694  0.050703  frozenset({S/stomach pain, S/headache, S/nausea})
```

```
695  0.050583   frozenset({S/joint pain, S/dizziness, S/nausea})
696  0.050504  frozenset({S/joint pain, S/brain fog, S/headac…
697  0.050145                   frozenset({S/migraine, S/nausea})

     support_count
693           2543
694           2542
695           2536
696           2532
697           2514
```

Task 2.6: Obtain association rules

From the frequent itemsets, obtain association rules with a minimum lift of 3.0 and sort the rules by lift in descending order.

Show the first couple of rows of the dataframe, limited to the columns *antecedents*, *consequents*, *support*, *confidence*, *lift*, and *kulczynski*.

[27]:

[27]:
```
                            antecedents                           consequents  \
0                frozenset({C/anxiety})  frozenset({C/depression, S/fatigue})
1  frozenset({C/depression, S/fatigue})                frozenset({C/anxiety})
2             frozenset({C/depression})     frozenset({C/anxiety, S/fatigue})
3     frozenset({C/anxiety, S/fatigue})              frozenset({C/depression})
4             frozenset({C/depression})                frozenset({C/anxiety})

    support  confidence      lift  kulczynski
0  0.057405    0.363200  4.388781    0.528431
1  0.057405    0.693661  4.388781    0.528431
2  0.057405    0.315639  4.047207    0.525850
3  0.057405    0.736061  4.047207    0.525850
4  0.109425    0.601667  3.806736    0.646997
```

Task 2.7: Filter association rules and evaluate an association rule

Filter out, i.e., remove, those association rules that contain a weather observation in the antecendents or consequents. How many rules have been removed? Show all rows of the filtered dataframe.

Finally, pick one of the remaing association rules and investigate the interestingness of that association rule. Write your textual answer in the markdown cell below.

[28]:

```
Number of removed association rules that contain a weather observation: 10234
```

[28]:
```
                              antecedents  \
0                    frozenset({C/anxiety})
1      frozenset({C/depression, S/fatigue})
```

```
2                 frozenset({C/depression})
3         frozenset({C/anxiety, S/fatigue})
4                 frozenset({C/depression})
5                    frozenset({C/anxiety})
6                  frozenset({S/back pain})
7                  frozenset({S/neck pain})
8                   frozenset({N/stressed})
9                      frozenset({N/tired})
10244               frozenset({S/diarrhea})
10245           frozenset({S/constipation})
10246              frozenset({S/dizziness})
10247   frozenset({S/joint pain, S/nausea})

                              consequents  antecedent support  \
0      frozenset({C/depression, S/fatigue})            0.158053
1                    frozenset({C/anxiety})            0.082757
2         frozenset({C/anxiety, S/fatigue})            0.181869
3                 frozenset({C/depression})            0.077989
4                    frozenset({C/anxiety})            0.181869
5                 frozenset({C/depression})            0.158053
6                  frozenset({S/neck pain})            0.123108
7                  frozenset({S/back pain})            0.131605
8                      frozenset({N/tired})            0.083634
9                   frozenset({N/stressed})            0.173392
10244           frozenset({S/constipation})            0.178518
10245               frozenset({S/diarrhea})            0.131226
10246   frozenset({S/joint pain, S/nausea})            0.161943
10247              frozenset({S/dizziness})            0.099531

       consequent support   support  confidence      lift  representativity  \
0                0.082757  0.057405    0.363200  4.388781               1.0
1                0.158053  0.057405    0.693661  4.388781               1.0
2                0.077989  0.057405    0.315639  4.047207               1.0
3                0.181869  0.057405    0.736061  4.047207               1.0
4                0.158053  0.109425    0.601667  3.806736               1.0
5                0.181869  0.109425    0.692327  3.806736               1.0
6                0.131605  0.060457    0.491089  3.731545               1.0
7                0.123108  0.060457    0.459382  3.731545               1.0
8                0.173392  0.050723    0.606487  3.497783               1.0
9                0.083634  0.050723    0.292534  3.497783               1.0
10244            0.131226  0.077989    0.436872  3.329161               1.0
10245            0.178518  0.077989    0.594315  3.329161               1.0
10246            0.099531  0.050583    0.312354  3.138247               1.0
10247            0.161943  0.050583    0.508216  3.138247               1.0

       leverage  conviction  zhangs_metric   jaccard  certainty  kulczynski
0      0.044325    1.440396       0.917096  0.312996   0.305746    0.528431
```

```
1      0.044325    2.748416       0.841812  0.312996   0.636154    0.528431
2      0.043221    1.347258       0.920288  0.283547   0.257752    0.525850
3      0.043221    3.099702       0.816602  0.283547   0.677388    0.525850
4      0.080680    2.113676       0.901210  0.474732   0.526891    0.646997
5      0.080680    2.659094       0.875718  0.474732   0.623932    0.646997
6      0.044255    1.706379       0.834783  0.311223   0.413964    0.475235
7      0.044255    1.622017       0.842951  0.311223   0.383484    0.475235
8      0.036222    2.100587       0.779279  0.245867   0.523943    0.449511
9      0.036222    1.295279       0.863897  0.245867   0.227966    0.449511
10244  0.054563    1.542764       0.851661  0.336518   0.351813    0.515593
10245  0.054563    2.024927       0.805300  0.336518   0.506155    0.515593
10246  0.034465    1.309494       0.813012  0.239856   0.236346    0.410285
10247  0.034465    1.704118       0.756662  0.239856   0.413186    0.410285
```

How do you interpret the interestingness of the rule you picked?

-> PUT YOUR ANSWER HERE <-

Task 2.8: Calculate the imbalance ratio

Add a column *imbalance_ratio* to the dataframe of the filtered rules (Task 2.7) indicating the imbalance ratio of each rule.

Show all rows of the dataframe, limited to the columns *antecedents*, *consequents*, and *imbalance ratio*.

Finally, judge the interestingness of the association rule you previously picked in Task 2.7 based on the imbalance ratio. Write your textual answer in the markdown cell below.

[29]:

```
```

[29]:

```
                                antecedents  \
0                    frozenset({C/anxiety})
1      frozenset({C/depression, S/fatigue})
2                 frozenset({C/depression})
3         frozenset({C/anxiety, S/fatigue})
4                 frozenset({C/depression})
5                    frozenset({C/anxiety})
6                  frozenset({S/back pain})
7                  frozenset({S/neck pain})
8                   frozenset({N/stressed})
9                      frozenset({N/tired})
10244               frozenset({S/diarrhea})
10245           frozenset({S/constipation})
10246              frozenset({S/dizziness})
10247   frozenset({S/joint pain, S/nausea})

                                consequents  imbalance_ratio
0      frozenset({C/depression, S/fatigue})         0.410549
1                    frozenset({C/anxiety})         0.410549
```

```
2         frozenset({C/anxiety, S/fatigue})         0.513103
3                 frozenset({C/depression})         0.513103
4                    frozenset({C/anxiety})         0.103323
5                 frozenset({C/depression})         0.103323
6                  frozenset({S/neck pain})         0.043742
7                  frozenset({S/back pain})         0.043742
8                      frozenset({N/tired})         0.435077
9                   frozenset({N/stressed})         0.435077
10244           frozenset({S/constipation})         0.204062
10245               frozenset({S/diarrhea})         0.204062
10246   frozenset({S/joint pain, S/nausea})         0.295942
10247              frozenset({S/dizziness})         0.295942
```

How do you judge the interestingness of the association rule you previously picked now?

-> PUT YOUR ANSWER HERE <-

Task 2.9: Perform a more detailed analysis of weather conditions, notes, etc.

Look more closely at the weather observations in the base dataframe. You will notice that some of the weather observations have values, e.g., 'temperature max' and 'humidity'. Employ binning to find more meaningful items than just 'W/temperature max' and 'W/humidity', e.g., 'W/temperature max 25-30' or 'W/humidity 50-60', before mining for frequent itemsets and meaningful association rules. Try out different binning strategies and see whether you can find interesting association rules, not just between weather observations but between weather observations and specifically symptoms. Look also at the notes and think about transformations that you could apply to possibly find more interesting association rules. Do you have any other ideas regarding the preprocessing?

Add additional code and markdown cells to document your analysis.

[29]:

```
```